Calcolatori Elettronici
Esercitazioni Assembler

M. Sonza Reorda – M. Monetti
M. Rebaudengo – R. Ferrero
L. Sterpone – M. Duni
morgana.duni@polito.it

Politecnico di Torino
Dipartimento di Automatica e Informatica

Informazioni generali

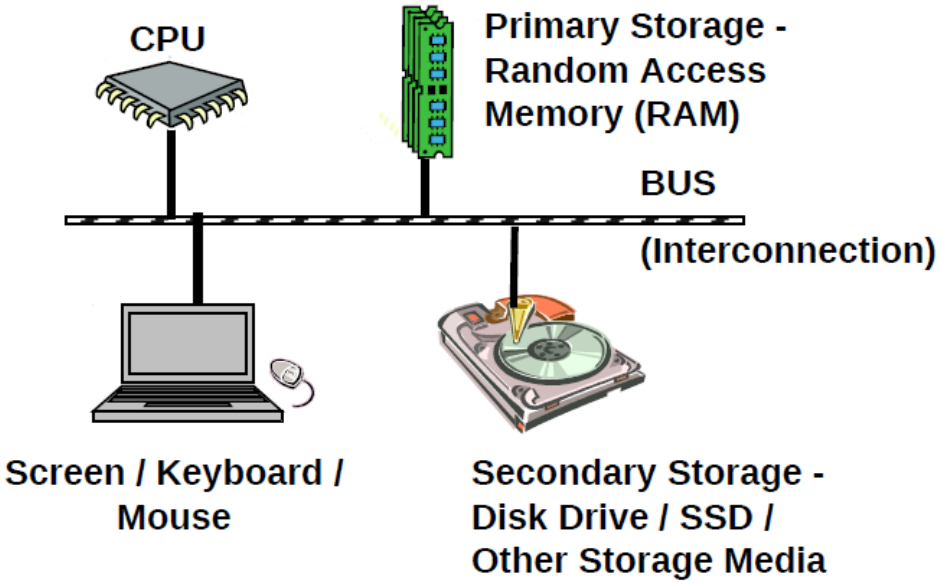
- LABINF
  - 1° piano, ingresso da C.so Castelfidardo lato ovest
  - è necessario avere un account
  - https://www.labinf.polito.it/
- 2 squadre
  - Lunedì 16:00 – 17:30 Squadra 1
  - Lunedì 17:30 – 19:00 Squadra 2
  - 1° laboratorio: 16 marzo 2026

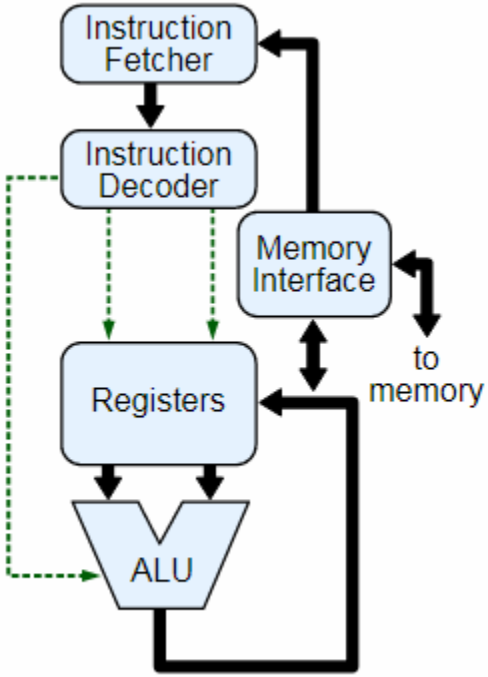
Organizzazione

- Il testo dell'esercitazione è caricato qualche giorno prima del turno della squadra 1
- La soluzione dell'esercitazione è caricata qualche giorno dopo il turno della squadra 2
- Partecipazione attiva:
  - in aula
  - a distanza: canale Telegram https://t.me/calceln2

Il calcolatore

- Schema dal punto di vista del programmatore in linguaggio Assembly

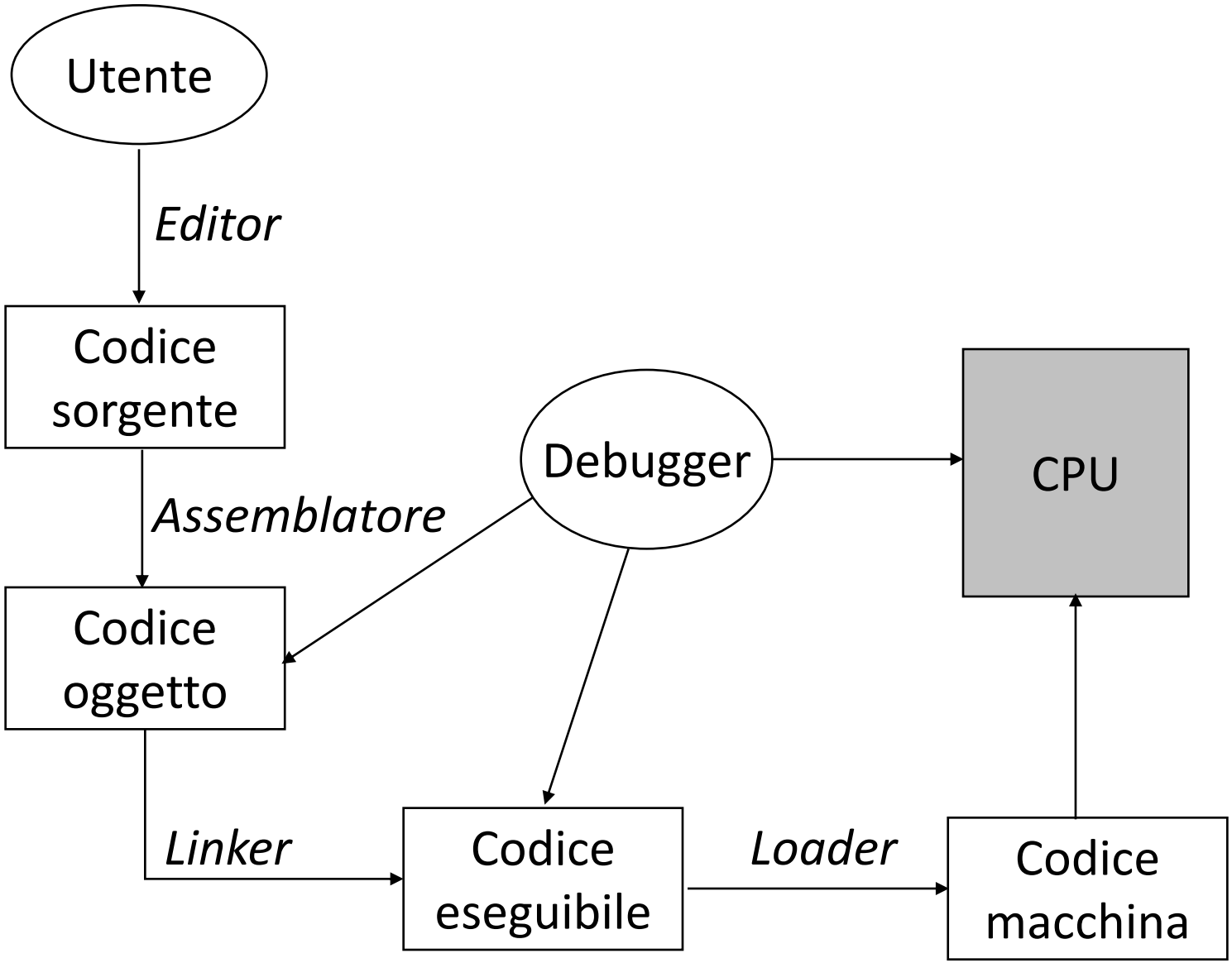




Architettura RISC-V: registri

Nome	Numero	Usage
`zero`	x0	il valore costante 0
`ra`	x1	indirizzo di ritorno della funzione
`sp`	x2	stack pointer
`gp`	x3	puntatore al segmento di variabili globali
`tp`	x4	puntatore all'area di memoria locale del thread
`t0-2`	x5-7	valori temporanei
`s0/fp`	x8	valore salvato / frame pointer
`s1`	x9	valore salvato
`a0-1`	x10-11	argomenti della funzione/ valori di ritorno
`a2-7`	x12-17	argomenti della funzione
`s2-11`	x18-27	valori salvati
`t3-6`	x28-31	valori temporanei

Ciclo di vita di un programma

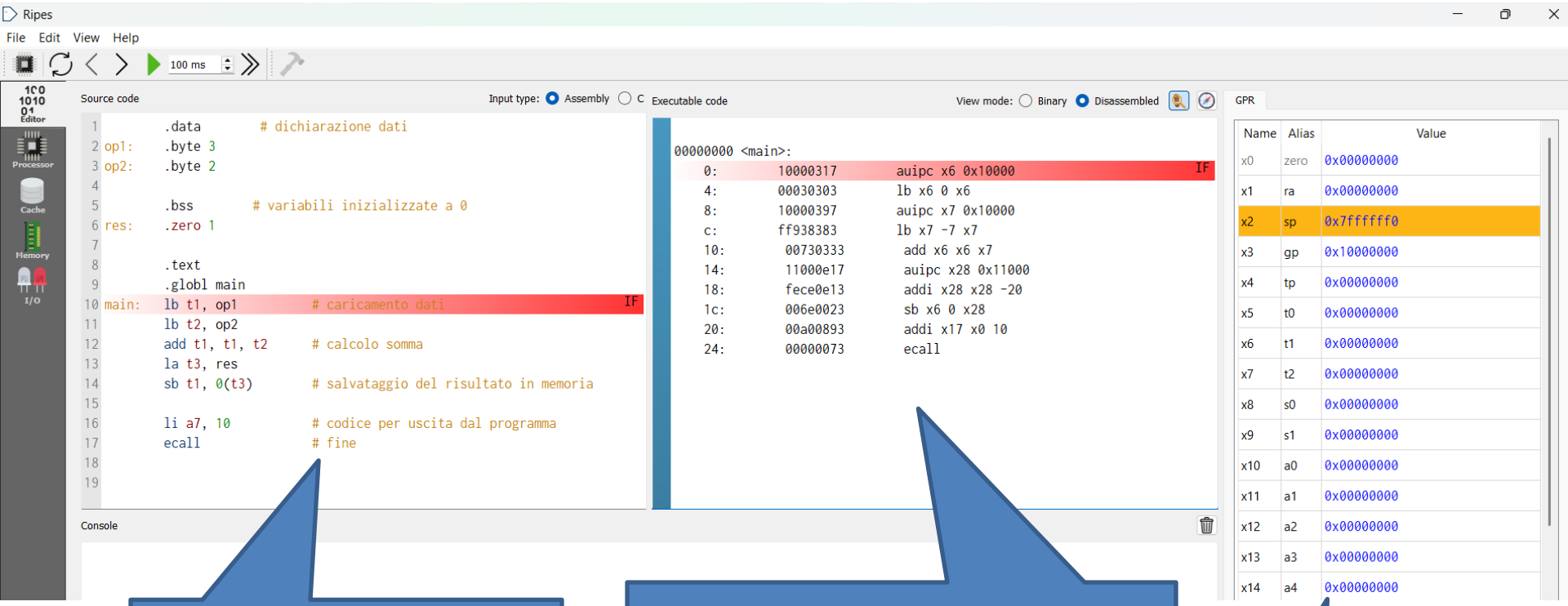




Ripes

- Simulatore di programmi per RISC-V
- Versione web: https://ripes.me/
- Versione desktop: https://github.com/mortbopet/Ripes
  - Codice sorgente
  - File binari già compilati per Windows, Linux e Mac
- Ben documentato: https://github.com/mortbopet/Ripes/blob/master/docs/introduction.md

Editor

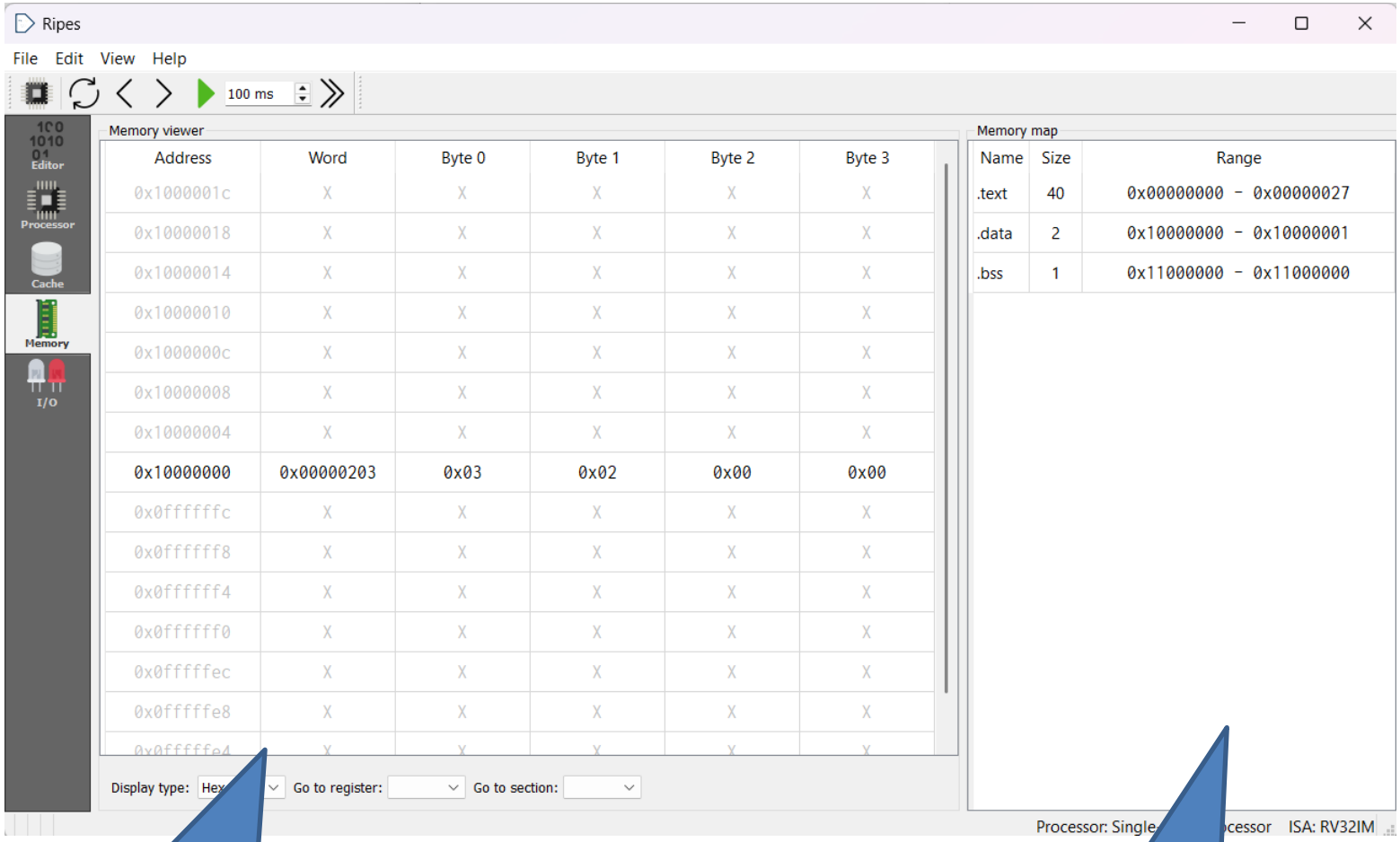


Codice sorgente

Istruzioni disassemblate (program viewer)

registri

Memoria



vista sull'intero spazio di indirizzamento

Intervallo dei segmenti

Lettura e scrittura di valori in memoria

```
      .data         # dichiarazione dati
op1:  .byte 3
op2:  .byte 2

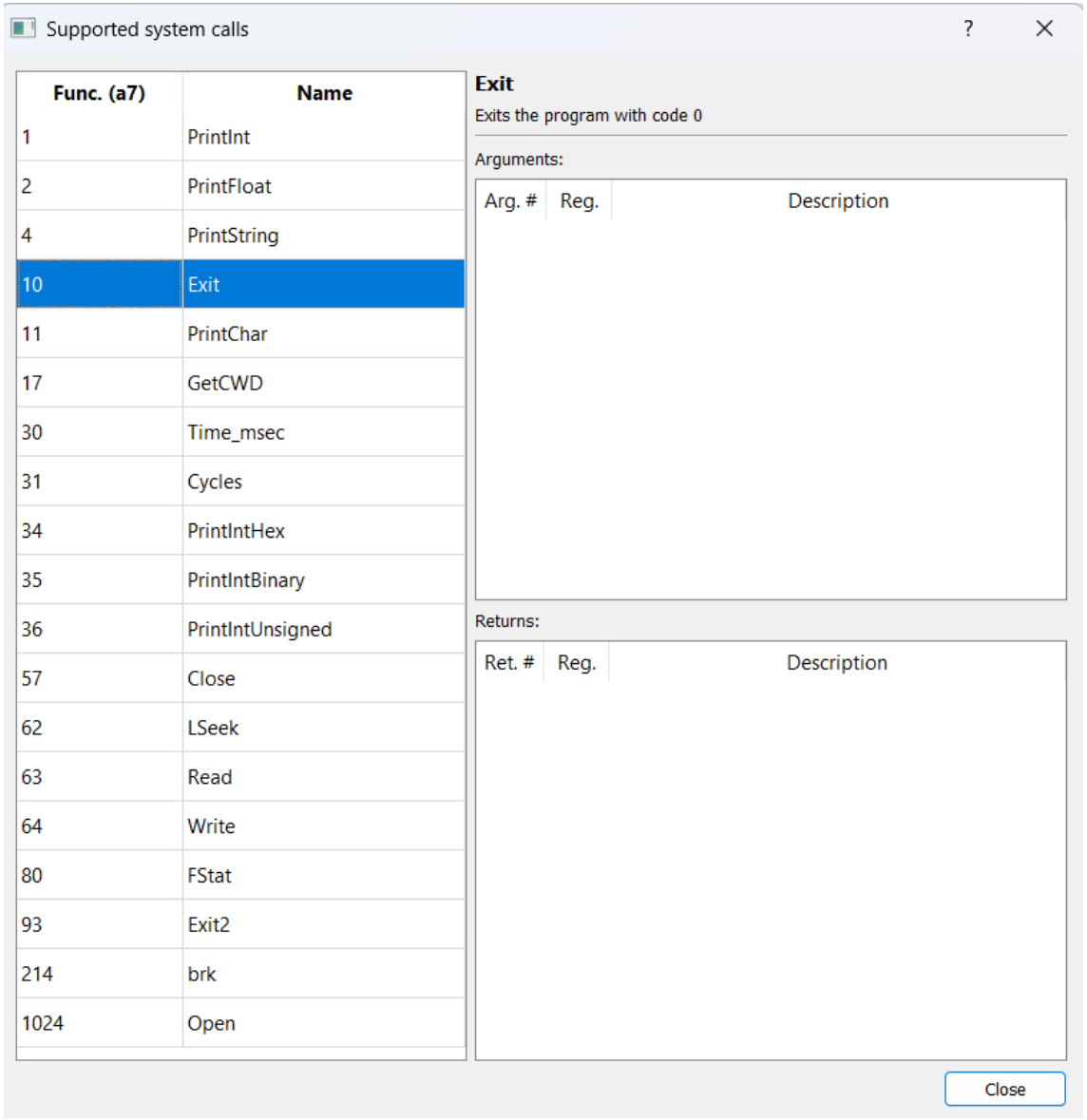
      .bss          # variabili inizializzate a 0
res:  .zero 1

      .text
      .globl main
main: lb t1, op1          # caricamento dati
      lb t2, op2
      add t1, t1, t2      # calcolo somma
      la t3, res
      sb t1, 0(t3)        # salvataggio in memoria
      li a7, 10   # codice per uscita dal programma
      ecall               # fine
```

System Calls

- RISC-V offre istruzioni `ecall` (environment call) per implementare system call
- Si tratta di richieste effettuate da un codice con privilegi inferiori (modalità utente) per eseguire codice con privilegi superiori (kernel)
- L'implementazione delle `ecall` dipende dall'ambiente di lavoro.

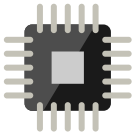
System Call supportate da Ripes



`ecall` in Ripes

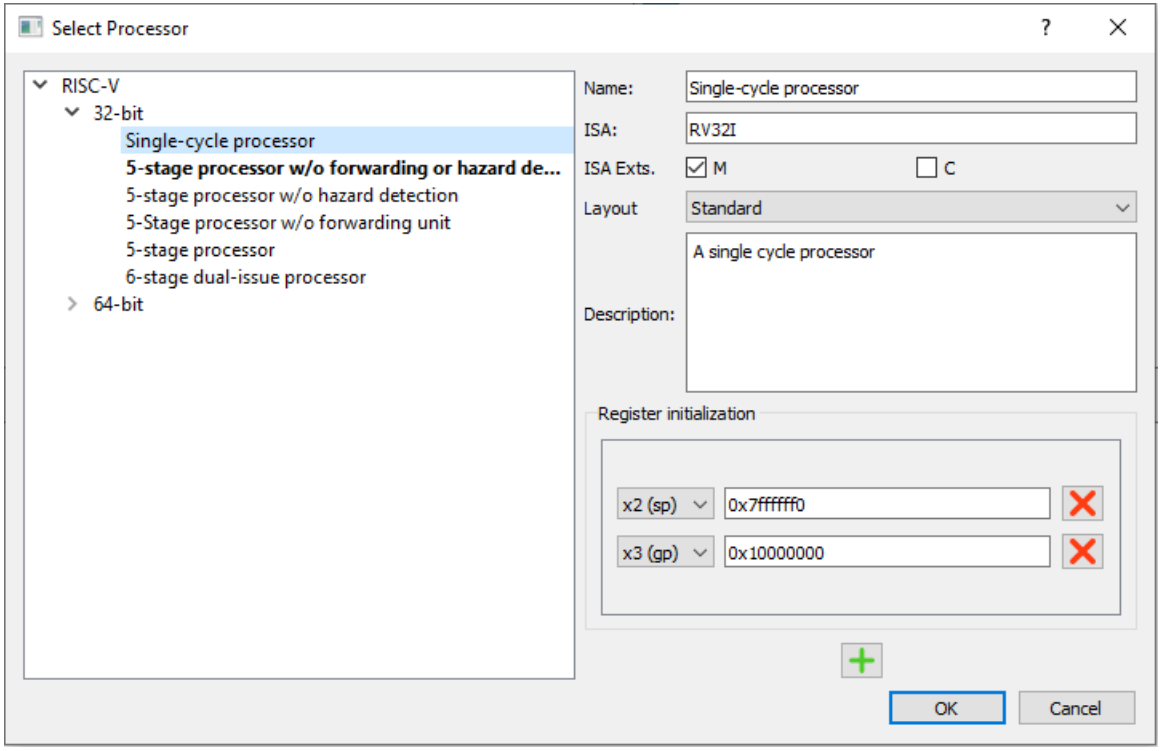
- Per usare una `ecall`
  - Si carica l'indice della ecall in a7
  - Si caricano eventuali argomenti in a0 e a1
  - Eventuali valori di ritorno vengono salvati in a0 e a1
- Questo è valido per Ripes: le `ecall` supportate in altri ambienti possono essere diverse

Comandi del simulatore

-  seleziona il processore
-  resetta il processore, ripartendo dall'inizio del programma
-  torna indietro, annullando l'ultima istruzione eseguita
-  esegue l'istruzione corrente
-  esegue tutte le istruzioni
-  esegue tutte le istruzioni senza aggiornare la GUI (modalità più veloce)

Selezione del processore

- Scegliere processore a 32 bit a ciclo singolo
  - la dimensione dei dati e degli indirizzi è 32 bit
  - ogni istruzione è completata in un singolo ciclo di clock



Input/Output da console (1)

```
        .data
msg1:   .string "Introduci il primo valore (una cifra sola): "
msg2:   .string "Introduci il secondo valore (una cifra sola): "
msg3:   .string "La somma dei due valori è "
        .bss
buffer: .zero 2

        .text
main:   # primo valore
        li a7, 4 # ecall 4 (print_str)
        la a0, msg1       # argomento: stringa
        ecall             # stampa la stringa

        li a7, 63         # ecall 63 (read)
        li a0, 0          # file descriptor 0: standard input (stdin)
        la a1, buffer
        li a2, 2          # massimo numero di caratteri da leggere
        ecall             # legge la stringa
        lb a0, 0(a1)
        addi t0, a0, -48  # 48 è il codice ASCII di '0'
```

Input/Output da console (2)

```
# secondo valore
li a7, 4          # ecall 4 (print_str)
la a0, msg2       # argomento: stringa
ecall             # stampa la stringa

li a7, 63         # ecall 63 (read)
li a0, 0          # file descriptor 0: standard input (stdin)
la a1, buffer
li a2, 2          # massimo numero di caratteri da leggere
ecall             # legge la stringa
lb a0, 0(a1)
addi t1, a0, -48

li a7, 4          # ecall 4 (print_str)
la a0, msg3       # argomento: stringa
ecall             # stampa la stringa

li a7,1           # ecall 1 (print_int)
add a0, t1, t0    # somma degli operandi
ecall
li a7, 10
ecall
```

Input/Output da console: note

```
msg1:.string "Introduci il primo valore (una cifra sola): "
```

- alloca un byte in più per il terminatore di stringa (NULL)
- NULL è un carattere ASCII non stampabile (con valore 0).
- è richiesto dalle system call che operano sulle stringhe (per funzionare correttamente).

```
li a2, 2      # massimo numero di caratteri da leggere
```

- il secondo carattere letto è l'invio
- non viene eseguito nessun controllo su tipo di carattere inserito (ovvero se corrisponde ad una cifra numerica)

Ricerca del valore minimo (1)

```
        .data
message_in:  .string "Inserire 5 numeri (di una cifra ciascuno)\n"
message_out: .string "I numeri inseriti sono: "
message_min: .string "\nIl valore minimo è "
separatore:  .string ", "

        .bss
buffer:  .zero 2
vettore: .zero 5

        .text
main:   la a0, message_in
        li a7, 4
        ecall

        la t0, vettore
        li t1, 5 # contatore numeri da leggere
        li a7, 63        # ecall 63 (read)
        la a1, buffer
        li a2, 2 # massimo numero di caratteri da leggere
```

Ricerca del valore minimo (2)

```
lettura: li a0, 0          # file descriptor 0: standard input (stdin)
         ecall             # legge la stringa
         lb a0, 0(a1)
         addi a0, a0, -48          # 48 è il codice ASCII di '0'
         sb a0, 0(t0)

         addi t0, t0, 1
         addi t1, t1, -1
         bne t1, x0, lettura

         la a0, message_out        # stampa i numeri letti
         li a7, 4
         ecall

         la t0, vettore
         li t1, 4
         lb t2, 0(t0)      # il minimo è inizializzato al primo valore

         li a7, 1          # ecall per print int
         mv a0, t2
         ecall
```

Ricerca del valore minimo (3)

```
scrittura: la a0, separatore
         li a7, 4
         ecall

         addi t0, t0, 1
         lb a0, 0(t0)
         bgtu a0, t2, stampa
         mv t2, a0          # nuovo minimo
stampa:  li a7, 1           # ecall per print int
         ecall
         addi t1, t1, -1
         bne t1, x0, scrittura

         la a0, message_min                 # stampa il valore minimo
         li a7, 4
         ecall

         li a7, 1
         mv a0, t2
         ecall

         li a7, 10
         ecall
```

Altri simulatori

- Venus: https://venus.kvakil.me/
- Creator: https://creatorsim.github.io/creator/
- emulsiV: https://eseo-tech.github.io/emulsiV/
- WebRISC-V: https://webriscv.altervista.org/
- RISC-V Console Simulator: https://github.com/UCDClassNitta/riscv-console
- QtRvSim–RISC-V: https://github.com/cvut/qtrvsim?tab=readme-ov-file
- RISC-V Interpreter: https://www.cs.cornell.edu/courses/cs3410/2019sp/riscv/interpreter/#memory-table
- rv8: https://michaeljclark.github.io/
- BRISCV: https://ascslab.org/research/briscv/simulator/manual.html
- Trireme: https://www.trireme-riscv.org/simulator/manual.html
- ALE: https://riscv-programming.org/ale/

Laboratorio

- Si prendano gli esempi di codice presentati in precedenza e quelli in “RISC-V Introduzione”.
- Inserire tali esempi di codice in Ripes, compilarli, eseguirli e analizzarne il comportamento in modalità di debug.



ESP32-C3 Super Mini

- Microcontrollore basato su RISC-V

- CPU: RV32IMC
- Frequenza: fino a 160 MHz
- WiFi + Bluetooth
- GPIO per controllare dispositivi



Ambiente di sviluppo

Editor di codice
VS Code
↓
Framework di sviluppo
ESP-IDF (Espressif IoT Development Framework)
↓
Toolchain RISC-V
Compiler e assembler (riscv32-esp-elf-gcc)
↓
Firmware binario
Programma compilato per ESP32
↓
Flash sulla scheda
Trasferimento via UART (porta COM)

Comandi principali:
idf.py build
idf.py flash
idf.py monitor

Progetto ESP-IDF: C e Assembly

Struttura progetto:

```
my_c3_project
  ├ CMakeLists.txt
  └ main
      ├ blink.c
      └
delay_cycles.S
```

Integrazione →

C code:

```
extern void
delay_cycles(uint32_t
cycles);
```

Assembly code:

```
delay_cycles:
      beqz a0, end
      addi a0, a0, -1
      j delay_cycles
end:
      ret
```

Il file .S contiene assembly RISC-V.

Il file C chiama la funzione assembly.